HPC 方向技术报告

光线追踪优化与 Mini WorkSteal 运行时

兰州大学超算队 2026 年秋季纳新 • 报告日期 2026-10-27

这份报告包含 HPC 方向必做的光线追踪题，以及选做的 Mini WorkSteal。我围绕两个问题展开：怎样减少光追中的重复求交并利用像素并行；工作窃取在什么条件下能够改善负载分配。正文选择主要实现和代表性对照，其他尝试与复现入口见文末链接中的配套附录。

项目	保留方案	主要结果
光线追踪	03、OpenMP 像素块并行、16 桶 SAH BVH、最终命中记录处理	Render time 中位数 1968.697 → 85.006 秒，耗时比 23.16；六份图像逐通道一致
WorkSteal	独立注册六类策略；保留 round_robin、immediate、single	同一优化二进制下，细粒度集中布局 off/on 的 epoch 时间比为 2.62-3.06；粗粒度未稳定获益

两项实验均在本机 Ubuntu 24.04 / WSL1 中编译和运行，CPU 为 i9-13900HX。光追记录来自 10 月 15-16 日，WorkSteal 记录截至 10 月 27 日。机器同时用于日常桌面工作，未独占频率、温度和后台负载，结果只代表已测条件。

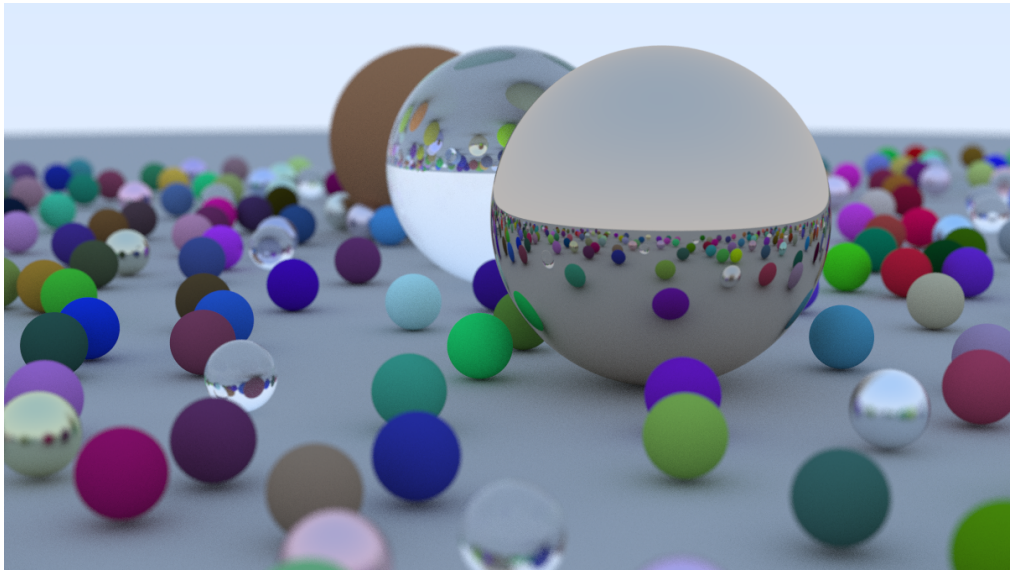


图1：同期正式对照中的最终版输出，原始文件为 P3 格式 PPM。PNG 仅作展示，图像质量以原始 PPM 检查结果为准。

1 光线追踪的分析与实现

1.1 从原版结构确定优化对象

原版逐像素采样，每条光线递归处理材质散射，场景求交时线性检查物体。我选择沿两条路径优化：把独立像素分配给多个线程，以及用空间结构排除不可能命中的物体。题目禁止修改的 main.cpp、common.hpp 和 math/color.* 保持不变；采样数、弹射深度、采样与散射分布、天空颜色和每像素随机数规则均保留。

1.2 按完整像素并行

每个像素有独立随机流，因此把同一像素的全部样本交给同一线程，保持像素内部的计算和随机数消耗顺序。OpenMP 只改变像素之间的执行顺序。最终以 16×8 像素块动态分配，chunk 为 1，本机采用 32 个线程。

线程将结果写入各自负责的像素位置，并行区结束后按原行列顺序输出。这样避免多个线程竞争同一个输出流，代价是增加约 18.54 MiB 的像素缓冲。缓冲分配、像素计算和顺序输出仍在 Render time 内；BVH 在场景初始化时建立，初始化耗时另外记录。

1.3 用 BVH 减少无效求交

BVH 用包围盒组织场景物体。光线未命中某个包围盒时，就可以跳过其中所有物体。我先比较中位数划分与 16 桶 SAH 划分；后者按子盒面积和物体数量估计划分代价，选择更有利的分组。叶子最多容纳 8 个物体，分桶退化时回退到中位数划分。

遍历优先访问入口更近的子树，再利用已找到的最近命中距离裁剪后续检查。求交保持 double 和原球体公式，并处理盒边界及等距命中的原物体优先级。空间剔除改变访问次序，仍需保证最近命中的选择与原版一致。

1.4 延后构造最终命中记录

原版每发现一个候选球体命中，就构造位置、法线和材质等完整记录；后续更近的命中会覆盖它。我将精确球体的路径分为距离计算和记录补全，在确定最终命中后再构造完整记录。未知物体和派生类型继续使用原接口。这项改动也包含精确类型的距离函数直调，实验没有单独拆分两者的贡献。

最终保留普通 O3 编译，未合入 LTO、PGO 或 SIMD。空间结构、并行和记录处理的局部对照分别保留，整套方案的加速比不能视为某一个方法的独立收益。

2 光线追踪的实验结果与取舍

2.1 正式配置与比较方法

正式配置为 1200×675、200 spp、最大深度 50、seed=42。主指标是题目输出的 Render time，包含渲染和文件输出。原版 O2 串行与最终版 O3 / SAH / OpenMP32 在同一批次各运行三次，顺序为 A1、B1、B2、A2、A3、B3，不额外预热，全部成功值均保留。

配对	原版 O2 / 秒	最终版 / 秒	原版 / 最终版
1	1897.516	84.857	22.36
2	1968.697	85.006	23.16
3	1993.332	85.197	23.40
中位数	1968.697	85.006	23.16

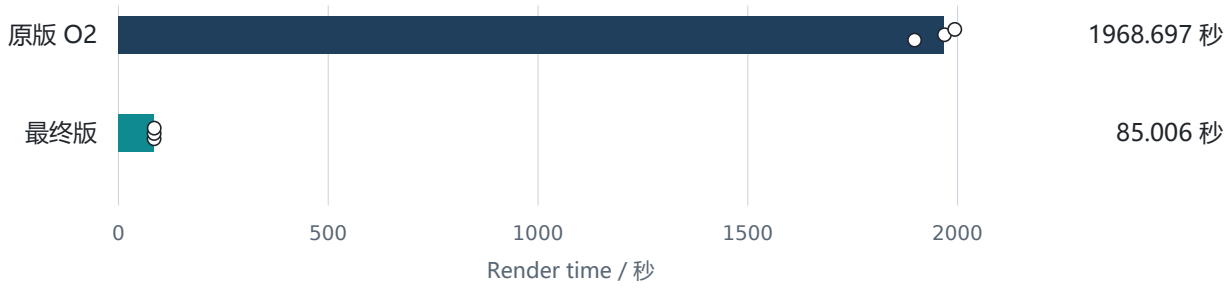


图2：条形表示中位数，圆点表示各次实际耗时，横轴从零起。比值包含编译、算法、线程数和绑定的共同变化；不能解释为纯 OpenMP 加速比，也不能用题目另一台 Xeon 的参考时间作分母。

2.2 正确性与代表性局部对照

六份正式图像与官方参考及同参数原版均无通道差异，MSE=0，PSNR 为无穷大，通过题目 45 dB 门槛。另做球体求交与记录差分检查，覆盖严格区间、等距命中、特殊方向、派生类型和场景生命周期。

局部对照	控制 / 秒	候选 / 秒	耗时降低
中位数 BVH → 16 桶 SAH	113.684	96.265	15.32%
立即构造 → 最终命中后构造	97.544	90.667	7.05%

两组局部对照每版各三次，仅可在同一行内比较，不能与整体对照连乘。后期候选的保留标准为 3/3 配对更快，且候选最大耗时低于控制最小耗时；这是工程选择规则，不是统计显著性检验。

2.3 测量限制

同一最终二进制在较早的独立验证中曾测得 17.435 秒，而本批约为 85 秒，波动原因未定位。因此主要成绩采用这批同期对照，不拼接跨批最快值。WSL1 下也未取得可靠的硬件热点采样，不能据此给出求交、缓存或同步各自的耗时占比。

3 Mini WorkSteal 的设计与实验安排

3.1 固定任务集与策略接口

WorkSteal 用于比较任务划分与动态调度。我采用持久线程池和带互斥锁的双端队列，任务集合在执行前固定，不支持执行中递归提交子任务。这样可以用剩余任务数判断结束，并保持策略间总工作量一致；锁和同步的成本也会计入测量。

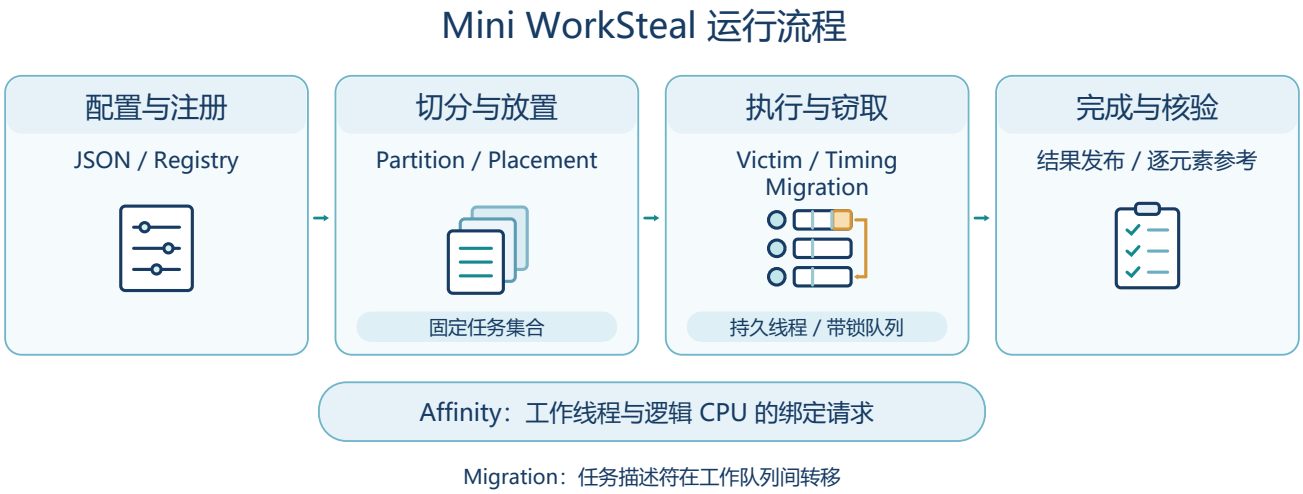


图3：策略提供调度决策，运行时处理同步和执行，应用独立核验结果。Migration 转移任务描述符；Affinity 提出逻辑 CPU 绑定请求，实际缓存驻留需另行验证。

策略	职责	代表实现
Partition	任务切分与粒度	fixed_grain、balanced_chunks
Placement	任务初始队列	按 home 或 round_robin 放置
Victim	选择窃取目标	round_robin、seeded_random
Timing	窃取与空闲等待时机	disabled、immediate、bounded_backoff
Migration	单次转移的任务数量	single、capped_half
Affinity	配置线程绑定	unbound、显式逻辑 CPU 列表

新增 test_reverse 的扩展检查只增加实现、注册单元和构建源清单，复用既有运行时。owner 从队尾取任务，thief 从队头窃取；任务写入独立结果槽位，再通过完成协议发布。检查覆盖恰好一次执行、重复调用、异常恢复与关闭。

3.2 工作负载和对照

均匀负载用于观察粒度与框架开销，偏斜负载用于观察动态分担，重复内存访问用于检查局部性。输入由 seed 和元素编号确定，改变 grain（每任务元素数）只改变分组。应用逐元素完整标量重算，小规模再用独立 Python 参考交叉检查。

实验加入串行、静态队列和集中队列对照。S3-off/on 共用队列、锁、任务顺序和完成协议，但 off 空闲时等待，on 空闲时 yield，差值同时包含空闲处理方式的影响。epoch 计线程池执行阶段，app 计含初始化和完整核验的应用总时间。

4 任务粒度与窃取收益

4.1 均匀负载中的开销对照

第一轮固定元素数 N=262144、基础迭代次数 K=1024，采用 4 个逻辑 worker，每个条件三次正式运行。表中为 epoch 中位数，单位毫秒。

grain	静态队列	集中队列	窃取关闭 off	窃取开启 on
1	209.39	212.17	209.59	210.54
64	210.53	205.91	209.86	206.67
4096	208.26	205.73	206.67	206.06

细粒度增加描述符、队列访问和完成通知；粗粒度减少可分配任务数。这组耗时差异较小，无法支持普遍最优的粒度选择，也无法分别量化队列锁等内部开销。off/on 仅提供整个调度路径的净差异。

4.2 偏斜布局与不可拆大任务

第十三轮固定同一优化二进制、16 个 worker 和 K=256，在两个新输入上，每个布局与粒度各做 5 对 off/on。集中布局 grain=8/256 的四个条件均为 5/5 对开启窃取更快，epoch 中位数时间比为 2.62-3.06。计算成本以整数混合运算 mix 的迭代次数计，不直接对应本机秒数。

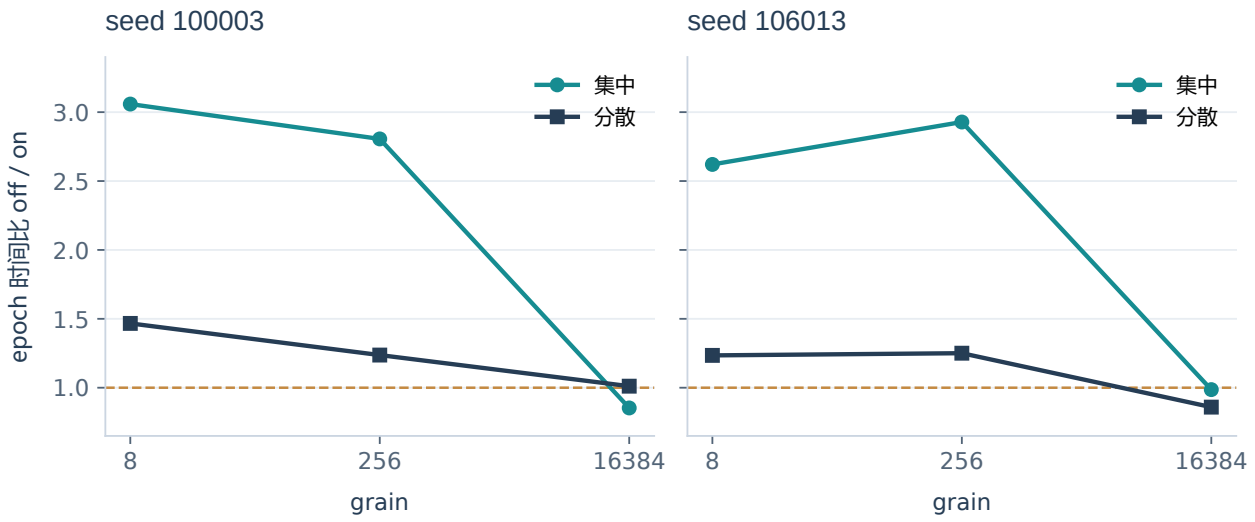


图4：时间比为 off 中位数 / on 中位数，大于 1 表示开启窃取更快；虚线为 1。每条件 n=5。粗粒度下，两种布局实际形成相同初始队列，因此不能将两组差异解释为布局效应。

grain=16384 时，两个输入分别只剩 4/6 个任务，最大任务占总成本单位的 62.5%/58.33%，窃取未稳定获益。当前运行时无法拆分已开始的任务，调整 victim 也无法增加这部分内部并行。分散布局中即使成本单位相等，完成时间仍可能不同；现有数据不足以确定其硬件原因，app 的变化也小于 epoch。

5 局部性与最终策略选择

5.1 局部性证据与不足

本机报告的 LLC 约为 36 MiB。实验覆盖 512 KiB、4.5 MiB 和 288 MiB 的每任务工作集，包含远小于与远大于 LLC 的两端。固定数据 home（归属标记）、访问遍数和预读，受控改变执行者，每条件三次正式计时。

每任务工作集	相对 LLC	方向0 foreign/home	方向1 foreign/home
512 KiB	1/72	1.043	1.113
4.5 MiB	1/8	1.025	0.986
288 MiB	8	0.973	1.070

表中为两个 home 方向各自的 foreign/home 耗时比，两方向没有一致趋势。六组开启窃取的动态诊断均未成功窃取；初始放置也可能使执行者不同于 home。WSL1 下未可靠核实物理驻留和缓存域，缺少 PMU 数据。因此，本项目尚未完成三种效应的充分分离，不能把这组结果解释为动态窃取的缓存代价接近零。

5.2 批量窃取的代表性尝试

批量迁移的假设是一次取得多个任务可以减少单位任务的窃取开销。第十四轮固定二进制、线程数、粒度和输入，比较 single 与 cap4/16/64；筛选选中 cap4，再用两个新输入确认。下表为集中布局，每输入 7 对正式运行。

输入 seed	epoch single→cap4 ms	更快对数	app single→cap4 ms
116009	3.7183 → 3.6328	3/7	31.6320 → 30.2063
122011	5.0438 → 5.1210	4/7	38.1532 → 41.2096

筛选阶段 epoch 下降约 22.7%，但确认中仅 3/7、4/7 对更快，第二输入 app 中位数上升 8.01%。诊断中成功 steals 减少、成本分配更均衡，同时描述符迁移总量增加。新输入结果未达到采用标准，因此保留 single；计数变化不足以单独证明性能收益。

5.3 保留配置与主要判断

调度层保留 fixed_grain、round_robin victim、immediate、single 和显式逻辑 CPU 掩码，初始队列由 placement 与 layout 共同决定。线程数和粒度按负载选择，见在线附录B。核验并行化和内存执行交错属于其他阶段的优化，分别列在在线附录A，未计作窃取收益。

这两个项目中，我依据实际对照选择保留方案。光追主要通过减少场景查询和像素并行降低耗时；WorkSteal 的结果则表明，调度收益取决于剩余可分配任务及其成本。现有结果不支持跨机器最优，也不支持把不同阶段、不同基线的倍数相乘。

补充材料与复现入口

详细附录、两个项目的源码、Linux 构建说明、配置和数据索引见配套 GitHub 的固定版本：

https://github.com/daicy2005/lzuhpc-2026-hpc/tree/338e37ed4f74fc1d4f189af98379e725fcc16645

连续对话记录（Prompt File）暂未上传。

AI 辅助包括资料搜集、备选方案梳理、部分代码编写和数据整理。